CS · 101 · Deep Dive Series · 2025

Algorithms & Logic

An in-depth exploration of algorithmic paradigms, data structure operations, formal language theory, computational complexity, and the mathematical foundations that underpin all of computer science.

PART I · Sorting · Graph Algorithms · Dynamic Programming · Divide & Conquer · Greedy · Backtracking

PART II · Boolean Algebra · Formal Languages · Turing Machines · P vs NP · Information Theory

Contents

PART I

Algorithms

An algorithm is a precise, finite sequence of instructions that transforms an input into a desired output. Mastery of algorithmic thinking — knowing which pattern to apply and why — is the defining skill that separates exceptional engineers from the rest.

01

Sorting Algorithms

Sorting is among the most studied problems in computer science — not because sorted output is always the end goal, but because an enormous number of problems reduce to sorting as a subroutine. The theoretical lower bound for comparison-based sorting is **O(n log n)**, proven via the decision-tree argument: there are n! possible orderings, and distinguishing them requires at least log■(n!) ≈ n log n comparisons.

Complexity Overview

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)	Yes
Tim Sort	O(n)	O(n log n)	O(n log n)	O(n)	Yes

Merge Sort — Divide and Conquer

Merge sort is the purest expression of divide and conquer for sorting. It recursively splits the array in half until single elements remain, then merges sorted halves back together. Its guarantee of O(n log n) in all cases — best, average, and worst — makes it ideal when predictable performance matters more than raw speed. The cost is O(n) auxiliary memory for the merge step.

```
# Merge Sort — always O(n log n), O(n) space
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid   = len(arr) // 2
    left  = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(L, R):
    result, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: result.append(L[i]); i += 1
        else:            result.append(R[j]); j += 1
    return result + L[i:] + R[j:]
```

Quick Sort — Pivot and Partition

Quick sort is typically faster than merge sort in practice despite an O(n²) worst case (triggered by sorted input with a naive pivot). It works by selecting a pivot, partitioning the array so all elements smaller than the pivot are to its left and larger to its right, then recursively sorting each partition. With random pivot selection, the expected recursion depth is O(log n), giving O(n log n) average time. It is cache-friendly and in-place (O(log n) stack space for recursion), which is why most standard libraries use quick sort or a hybrid like Timsort (Python, Java) or Introsort (C++).

```
# Quick Sort — O(n log n) average, O(n^2) worst (naive pivot)
def quick_sort(arr, lo=0, hi=None):
    if hi is None: hi = len(arr) - 1
    if lo < hi:
        p = partition(arr, lo, hi)
        quick_sort(arr, lo, p - 1)
        quick_sort(arr, p + 1, hi)

def partition(arr, lo, hi):
    pivot = arr[hi]   # naive: last element
    i = lo - 1
    for j in range(lo, hi):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[hi] = arr[hi], arr[i+1]
    return i + 1
```

Non-Comparison Sorts: Breaking O(n log n)

The O(n log n) lower bound applies only to comparison-based algorithms. If we know something about the keys, we can do better. **Counting sort** counts occurrences of each integer key in range [0, k] and reconstructs the sorted array in O(n + k) time. **Radix sort** extends this to multi-digit numbers by sorting digit by digit (least significant to most), running in O(nk) where k is the number of digits. These are the algorithms of choice for fixed-width integers, such as sorting network packets by port

number or sorting fixed-length strings.

02

Graph Algorithms

Graphs model pairwise relationships and are among the most versatile structures in computer science. Graph algorithms underpin GPS navigation, social network analysis, compiler dependency resolution, circuit design, and machine learning. A graph G = (V, E) consists of a set of vertices V and edges E.

BFS — Breadth-First Search

BFS explores all neighbors of the current node before moving to their neighbors — layer by layer. It uses a queue (FIFO). The critical guarantee: the first time BFS reaches any node, it has done so via the fewest possible edges. This makes BFS the correct algorithm for shortest paths in unweighted graphs. Time and space complexity are both O(V + E). Applications include shortest path in unweighted graphs, level-order tree traversal, web crawling, and social network distance.

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue   = deque([start])
    dist    = {start: 0}
    while queue:
        node = queue.popleft()
        for nbr in graph[node]:
            if nbr not in visited:
                visited.add(nbr)
                dist[nbr] = dist[node] + 1
                queue.append(nbr)
    return dist   # shortest distances from start
```

DFS — Depth-First Search

DFS dives as deep as possible down one branch before backtracking. It uses a stack (either explicit or the call stack via recursion). DFS is the right tool for cycle detection, topological sorting (Kahn's algorithm or DFS post-order), finding strongly connected components (Tarjan's or Kosaraju's), generating mazes, and solving constraint-satisfaction problems. O(V + E) time and space.

Dijkstra's Algorithm

Dijkstra finds shortest paths from a source in a weighted graph with **non-negative edge weights**. It uses a min-priority queue. The key invariant: once a node is popped from the queue, its shortest distance is finalized. Time complexity is O((V + E) log V) with a binary heap, or O(V^2) with a plain array (better for dense graphs). Dijkstra fails with negative weights — Bellman-Ford handles those at O(VE).

```python
import heapq

def dijkstra(graph, src):
    dist = {node: float('inf') for node in graph}
    dist[src] = 0
    heap = [(0, src)]   # (distance, node)
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]: continue   # stale entry
        for v, w in graph[u]:      # (neighbor, weight)
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist
```

A* Search

A* extends Dijkstra with a heuristic h(n) — an estimate of the remaining cost from node n to the goal. The priority queue sorts by f(n) = g(n) + h(n), where g(n) is the cost from start to n. If h is *admissible* (never overestimates) and *consistent* (h(n) ≤ cost(n,n') + h(n')), A* finds the optimal path and typically explores far fewer nodes than Dijkstra by focusing toward the goal. It is the standard algorithm for pathfinding in games, robotics, and mapping software.

Topological Sort

A topological ordering of a DAG (directed acyclic graph) is a linear ordering of vertices such that for every directed edge u→v, u comes before v. Topological sort has O(V + E) complexity and is the foundation of build systems (Make, Gradle), course prerequisite scheduling, and compiler dependency resolution. It is impossible if the graph contains a cycle.

03

Dynamic Programming

Dynamic programming (DP) is one of the most powerful algorithmic paradigms. It applies when a problem has two properties: **overlapping subproblems** (the same sub-computation appears many times) and **optimal substructure** (the optimal solution to the whole contains optimal solutions to its parts). DP trades exponential time for polynomial time by storing previously computed results.

TWO DP APPROACHES

Top-down (memoization): write the naive recursion, then add a cache. Bottom-up (tabulation): fill a table iteratively from base cases to the answer. Both have the same asymptotic complexity; bottom-up avoids recursion overhead.

Fibonacci: The Canonical Example

```
# Naive: O(2^n) — recomputes same values exponentially
def fib_naive(n):
    if n <= 1: return n
    return fib_naive(n-1) + fib_naive(n-2)

# Memoized: O(n) — each value computed exactly once
def fib(n, memo={}):
    if n <= 1: return n
    if n not in memo:
        memo[n] = fib(n-1) + fib(n-2)
    return memo[n]

# Bottom-up: O(n) time, O(1) space
def fib_iter(n):
    a, b = 0, 1
    for _ in range(n): a, b = b, a + b
    return a
```

0/1 Knapsack Problem

Given n items each with a weight w■ and value v■, and a knapsack of capacity W, maximize total value without exceeding W. Each item can be taken at most once. Define dp[i][w] = maximum value using first i items with capacity w. Recurrence: dp[i][w] = max(dp[i-1][w], dp[i-1][w-w■]+v■) if w■ ≤ w. Time and space: O(nW).

```python
def knapsack(weights, values, W):
    n  = len(weights)
    dp = [[0] * (W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            dp[i][w] = dp[i-1][w]
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w],
                    dp[i-1][w - weights[i-1]] + values[i-1])
    return dp[n][W]
```

Classic DP Problems Reference

Problem	State	Complexity	Application
Longest Common Subseq.	dp[i][j]	O(mn)	Diff tools, DNA alignment
Edit Distance	dp[i][j]	O(mn)	Spell checkers, NLP
Coin Change	dp[w]	O(nW)	Currency, resource alloc.
Matrix Chain Mult.	dp[i][j]	$O(n^3)$	ML, linear algebra
Longest Inc. Subseq.	dp[i]	O(n log n)	Stock analysis, biology
Floyd-Warshall	dp[k][i][j]	$O(V^3)$	All-pairs shortest paths

04

Divide & Conquer

Divide and conquer splits a problem into **independent** subproblems (unlike DP, which handles overlapping ones), solves each recursively, and combines the results. The Master Theorem analyzes recurrences of the form T(n) = aT(n/b) + O(n■): if d > log_b(a) then O(n■); if d = log_b(a) then O(n■ log n); if d < log_b(a) then O(n^(log_b a)).

Strassen's Matrix Multiplication

Naive matrix multiplication of two n×n matrices requires O(n³) operations (n² dot products each taking O(n) time). Strassen (1969) showed that four quadrant sub-matrices can be multiplied using only 7 recursive multiplications instead of 8, yielding T(n) = 7T(n/2) + O(n²). By the Master Theorem, this is O(n^(log■ 7)) ≈ O(n²·■■■). Though impractical for small n due to constants, it was the first proof that matrix multiplication could be done faster than O(n³), spawning decades of research.

Fast Fourier Transform (FFT)

The FFT computes the Discrete Fourier Transform of n points in O(n log n) instead of the naive O(n²). It works by exploiting the even/odd structure of the DFT recursively (Cooley-Tukey algorithm). The FFT enables fast polynomial multiplication (multiply two degree-n polynomials in O(n log n) instead of O(n²)), which in turn powers fast large-integer multiplication, signal processing, audio compression (MP3), image compression (JPEG), and convolution in neural networks.

05

Greedy Algorithms

A greedy algorithm makes the locally optimal choice at each step without reconsidering prior decisions. It works when the problem has the **greedy-choice property** (local optima build toward a global optimum) and **optimal substructure**. Proving correctness typically requires an exchange argument: show that any solution that deviates from the greedy choice can be transformed into the greedy solution without getting worse.

Huffman Encoding

Huffman encoding builds an optimal prefix-free variable-length code for lossless data compression. The algorithm: (1) treat each symbol as a single-node tree with weight equal to its frequency; (2) always merge the two lowest-weight trees into a new tree; (3) repeat until one tree remains. The resulting binary tree assigns shorter codes to more frequent symbols. This greedy strategy provably produces the optimal code — no prefix-free encoding can achieve a shorter expected length. Huffman coding is used in ZIP, DEFLATE, JPEG, and MP3.

Minimum Spanning Tree (Kruskal & Prim)

A minimum spanning tree of a connected weighted graph connects all vertices with minimum total edge weight, using exactly V-1 edges. **Kruskal's** algorithm sorts all edges by weight and greedily adds each edge if it does not form a cycle (using Union-Find), running in O(E log E). **Prim's** algorithm grows the MST from a start vertex, always adding the cheapest edge to an unvisited vertex, running in O(E log V) with a priority queue.

Activity Selection & Interval Scheduling

Given n activities each with a start and end time, select the maximum number of non-overlapping activities. The greedy choice: always pick the activity that finishes earliest. This provably maximizes the number of activities selected. The algorithm sorts activities by finish time and greedily selects compatible ones — O(n log n) for the sort, O(n) for the selection.

> **WHEN GREEDY FAILS**
>
> Greedy algorithms fail when local choices lead to global suboptima. Classic failure: coin change with denominations {1, 3, 4} and target 6. Greedy picks 4+1+1=3 coins; DP finds 3+3=2 coins. Always verify the greedy-choice property before applying this paradigm.

06

Backtracking & Branch-Bound

Backtracking is a systematic search that builds candidates incrementally and abandons a partial candidate ('backtracks') as soon as it is determined that it cannot lead to a valid or optimal solution. It transforms an exponential search space into a tree and prunes branches early.

N-Queens Problem

Place N queens on an N×N chessboard so no two queens attack each other. Backtracking places queens column by column, checking validity after each placement and backtracking if no valid row exists. For N=8, there are 92 solutions out of 8■ = 16 million brute-force possibilities.

```
def solve_n_queens(n):
    results = []
    cols = set(); diag1 = set(); diag2 = set()

    def backtrack(row, board):
        if row == n:
            results.append(board[:]); return
        for col in range(n):
            if col in cols or (row-col) in diag1 or (row+col) in diag2:
                continue   # prune: queen attacks
            cols.add(col); diag1.add(row-col); diag2.add(row+col)
            board.append(col)
            backtrack(row + 1, board)
            board.pop()    # undo choice
            cols.discard(col); diag1.discard(row-col); diag2.discard(row+col)

    backtrack(0, [])
    return results
```

Branch and Bound

Branch and bound extends backtracking for optimization problems by maintaining a bound on the best solution found so far. Any branch whose optimistic upper bound (for maximization) or lower bound (for minimization) is worse than the current best is pruned entirely. This makes the Travelling Salesman Problem (TSP) tractable for moderate instances — modern TSP solvers using B&B have solved instances with tens of thousands of cities.

PART II

Logic & Theory

Theoretical computer science asks the deepest questions: What can be computed? How efficiently? What are the fundamental limits of algorithms and machines? These answers draw from mathematics, logic, and formal language theory — and they underpin compilers, cryptography, AI, and the philosophy of mind.

07

Boolean Algebra

Boolean algebra is the mathematical skeleton of every digital circuit. George Boole formalized it in 1847 for propositional logic; Claude Shannon connected it to electrical relay circuits in his 1937 MIT master's thesis — widely considered the most impactful master's thesis ever written — laying the foundation for all modern digital hardware.

Core Operations

Operation	Symbol	Truth Table	Gate Description
AND	&	1,1→1; else→0	Both inputs must be 1
OR	|	0,0→0; else→1	At least one input is 1
NOT	~	0→1; 1→0	Inverts the input (unary)
XOR	^	same→0; diff→1	Exactly one input is 1
NAND	~&	NOT(AND)	Universal gate (can build any circuit)
NOR	~|	NOT(OR)	Also a universal gate

Key Laws and De Morgan's Theorem

Boolean algebra obeys a rich set of laws: commutativity, associativity, distributivity, identity (A AND 1 = A), annihilation (A AND 0 = 0), idempotence (A AND A = A), and complement (A AND NOT A = 0). De Morgan's Laws are among the most useful:

```
NOT(A AND B)  =  (NOT A) OR  (NOT B)   -- De Morgan 1
NOT(A OR  B)  =  (NOT A) AND (NOT B)   -- De Morgan 2

-- Example: negate a compound condition
-- Original:  if (x > 0 and y > 0)
-- Negated:   if (x <= 0 or y <= 0)
```

CNF, DNF, and the SAT Problem

Every Boolean function can be written in **Conjunctive Normal Form (CNF)** (AND of OR-clauses, e.g., (A∨B)∧(¬A∨C)) or **Disjunctive Normal Form (DNF)** (OR of AND-clauses). The **Boolean Satisfiability Problem (SAT)** asks: given a CNF formula, does an assignment of variables exist that makes it true? SAT was the first problem proven NP-complete (Cook-Levin, 1971). Despite this, modern SAT solvers (DPLL, CDCL) handle formulas with millions of variables and are used in hardware verification (EDA), AI planning, and cryptanalysis.

08

Formal Language Theory

Formal language theory asks: what kinds of strings can a given computational model recognize or generate? Noam Chomsky's 1956 hierarchy classifies languages into four levels, each requiring increasingly powerful machines. This framework directly informs compiler design, natural language processing, and the limits of pattern matching.

The Chomsky Hierarchy

Type	Language Class	Machine	Example
3	Regular	Finite Automaton (DFA/NFA)	a*b, identifiers, IP addresses
2	Context-Free	Pushdown Automaton (PDA)	a^n b^n, programming language syntax
1	Context-Sensitive	Linear Bounded Automaton	a^n b^n c^n
0	Rec. Enumerable	Turing Machine	Anything computable (and more)

Regular Languages and Finite Automata

A deterministic finite automaton (DFA) has a finite set of states, reads input symbols one at a time, and ends in an accept or reject state. Regular expressions — which you use in grep, Python's re module, and text editors — exactly describe the class of regular languages. The **Pumping Lemma for regular languages** proves that certain languages are not regular: if a language is regular and a string is long enough, it contains a repeatable substring. The language {a■ b■ | n ≥ 0} fails this test and is therefore not regular — a finite automaton cannot count.

Context-Free Languages and Grammars

Context-free grammars (CFGs) use production rules of the form A → α, where A is a non-terminal and α is any string of terminals and non-terminals. The language {a■b■} is context-free (grammar: S → aSb | ε). Programming language syntax is typically context-free, which is why parsers use CFG-based techniques: LL(k) parsers (top-down, used in recursive descent), LR(k) parsers (bottom-up, used in yacc/bison), and Earley's O(n³) algorithm for any CFG. However, type checking and semantic analysis exceed CFG power — these are handled by attribute grammars or separate analysis phases.

```
-- Context-Free Grammar for arithmetic expressions
E  ->  E '+' T  |  T
T  ->  T '*' F  |  F
F  ->  '(' E ')' | number

-- This grammar encodes operator precedence:
-- * binds tighter than + because T is deeper than E
```

09

Turing Machines & Computability

Alan Turing's 1936 paper 'On Computable Numbers, with an Application to the Entscheidungsproblem' introduced the Turing machine and, simultaneously, proved that certain problems are forever beyond the reach of any algorithm. It is one of the most consequential papers ever written.

The Turing Machine Model

A Turing machine consists of: (1) an **infinite tape** divided into cells, each holding a symbol; (2) a **read/write head** that can move left or right; (3) a **finite state register**; and (4) a **transition table** that specifies, for each (state, symbol) pair, what to write, which direction to move, and what state to transition to. Despite its simplicity, a Turing machine can simulate any algorithm that runs on a modern computer.

The Church-Turing Thesis

The Church-Turing thesis (not a theorem, but a deeply supported conjecture) states: any function that is 'effectively computable' by any physical device or process can be computed by a Turing machine. This means that Python, Java, C, and any other Turing-complete language can all solve exactly the same set of problems — they differ only in convenience and efficiency, not capability.

The Halting Problem — Undecidability

The Halting Problem asks: given a program P and input I, will P halt on I? Turing proved in 1936 that no algorithm can solve this for all (P, I) pairs. The proof is an elegant diagonal argument:

```
-- Proof by contradiction
Suppose HALTS(P, I) exists — returns True if P halts on I.

Construct D(P):
    if HALTS(P, P): loop forever   # if P halts on itself, don't halt
    else:           halt           # if P doesn't halt on itself, halt

Now run D(D):
    if HALTS(D, D) = True  -> D loops forever -> contradiction
    if HALTS(D, D) = False -> D halts         -> contradiction

Therefore HALTS cannot exist.  QED.
```

From the Halting Problem, **Rice's Theorem** follows: any non-trivial semantic property of programs is undecidable. You cannot write a general-purpose tool that determines 'does this program ever print hello?' or 'are these two programs equivalent?' for arbitrary inputs. This has profound implications for software verification and security analysis.

10

Complexity Classes & P vs NP

Beyond asking what can be computed, complexity theory asks: among the problems that can be computed, how efficiently? Complexity classes organize problems by the resources (time, space, randomness) required to solve them.

The Core Classes

Class	Definition	Key Examples
P	Solvable in polynomial time O(n^k)	Sorting, shortest path, primality (AKS)
NP	Solution verifiable in polynomial time	SAT, TSP (decision), Clique, Subset Sum
NP-complete	Hardest problems in NP; any NP reduces to them	SAT, 3-COLOR, Hamiltonian Cycle
NP-hard	At least as hard as NP-complete	TSP (optimization), Halting Problem
co-NP	Complement of NP (no-instances verif.)	Tautology, primality (historically)
PSPACE	Solvable in polynomial space	Generalized chess, QBF
EXP	Solvable in exponential time	Optimal chess, some circuit problems

The P vs NP Question

Does P = NP? This is the most famous open problem in computer science and one of the seven Millennium Prize Problems (with a $1 million prize). Informally: can every problem whose solution can be quickly verified also be quickly solved? Most researchers believe P ≠ NP, because P = NP would imply that creativity requires no more effort than verification — a philosophically startling collapse. It would mean that finding a proof is no harder than checking one, that cracking encryption is as easy as verifying a password, and that drug discovery and protein folding could be solved optimally in polynomial time.

NP-Completeness and Reductions

A **polynomial-time reduction** from problem A to problem B means: if you can solve B in polynomial time, you can solve A in polynomial time (by transforming A-instances into B-instances). A problem is **NP-complete** if it is in NP and every NP problem reduces to it. The first NP-complete problem was Boolean SAT (Cook-Levin, 1971). Thousands of NP-complete problems are now known, all connected by reductions. Proving a problem NP-complete is the standard way to justify using approximation algorithms or heuristics instead of searching for an exact polynomial-time solution.

> **FAMOUS NP-COMPLETE PROBLEMS**
>
> 3-SAT → Independent Set → Vertex Cover → Clique → Graph Coloring. Also: Hamiltonian Cycle, Subset Sum, Knapsack, TSP (decision), Bin Packing, Sudoku (generalized), Minesweeper consistency, and most scheduling problems.

11

Information Theory

Claude Shannon's 1948 paper 'A Mathematical Theory of Communication' founded information theory and established the mathematical basis for digital communication and compression. Shannon quantified information for the first time, answering: how much data can a noisy channel carry, and how compactly can a message be encoded?

Entropy — Measuring Information

Shannon entropy H(X) measures the average information content (surprise) of a random variable X with probability distribution {p(x)}:

```
H(X) = -SUM over x of [ p(x) * log2(p(x)) ]   (in bits)

-- Examples:
Fair coin (p=0.5, p=0.5):     H = 1.0 bit
Biased coin (p=0.9, p=0.1):   H = 0.469 bits
Fair 4-sided die:             H = 2.0 bits
Certain event (p=1.0):        H = 0 bits (no surprise)
```

Entropy is not just an abstract measure — it is the theoretical minimum number of bits needed per symbol to encode a message without loss. No lossless compression scheme can compress a source below its entropy. Huffman coding approaches the entropy limit; arithmetic coding essentially achieves it.

Channel Capacity — Shannon's Noisy Channel Theorem

Every noisy communication channel has a maximum rate C (in bits per channel use) at which information can be transmitted with arbitrarily low error probability. For an additive white Gaussian noise (AWGN) channel:

```
C = B * log2(1 + S/N)   -- Shannon-Hartley Theorem

Where:
  B   = bandwidth in Hz
  S/N = signal-to-noise ratio (linear, not dB)

-- A channel with 1 MHz bandwidth and SNR = 7 (8.5 dB):
C = 1,000,000 * log2(1 + 7) = 3,000,000 bits/sec = 3 Mbps
```

Shannon's theorem is existential — it guarantees such codes exist but does not construct them. Finding practical codes that approach capacity took decades: turbo codes (1993) and LDPC codes (rediscovered 1996) finally came within fractions of a decibel of the Shannon limit. These codes are now used in 4G/5G, deep-space communication, and Wi-Fi.

Kolmogorov Complexity

Kolmogorov complexity K(x) of a string x is the length of the shortest program that produces x. It measures the inherent randomness or compressibility of a string — a string is random if and only if it cannot be significantly compressed (K(x) ≈ |x|). Kolmogorov complexity is uncomputable (it reduces to the Halting Problem), but it provides a rigorous theoretical framework

for randomness, learning theory, and the foundations of data compression.

This guide covers the core algorithmic paradigms and theoretical foundations of computer science. For implementation practice, work through problems on LeetCode, Codeforces, or CLRS (Introduction to Algorithms, Cormen et al.).